TABLE OF CONTENTS

1. INTRODUCTION . . . . . **Page 2**

2. OBJECTIVES . . . . . **Page 3**

3. PROBLEM STATEMENT . . . . . **Page 4**

4. USE CASE DIAGRAM . . . . . **Page 5**

5. SYSTEM REQUIREMENTS . . . . . **Page 6**

6. ARCHITECTURE DESIGN . . . . . **Page 7**

7. METHODOLOGY . . . . . **Page 8**

8. FUTURE SCOPE . . . . . **Page 9**

9. CONCLUSION . . . . . **Page 10**

Project Title:	Sentinel AI — Real-Time Financial Fraud Intelligence
Project Category:	Academic B.Tech Computer Science Project
Core Focus:	Real-Time Transaction Monitoring & Explainable Fraud Detection
Technology Stack:	Apache Kafka, Apache Spark, Python, PostgreSQL/Supabase, Next.js
Document Scope:	Comprehensive System Overview, Design Specifications & Verification

INTRODUCTION

Sentinel AI is an academic computer science project designed to monitor financial card transactions in real time. The system automatically inspects payment activities as they happen, identifies unusual spending patterns, and flags potentially fraudulent transactions for human investigators to review. Developed as a capstone project for the Bachelor of Technology in Computer Science, Sentinel AI demonstrates how modern big data technologies and machine learning algorithms can be combined to solve a critical real-world problem.

Financial fraud represents billions of dollars in losses across the globe every year. With the rapid expansion of electronic commerce, mobile payments, and contactless cards, millions of financial records are processed every hour. Criminals frequently attempt to exploit stolen card numbers, leaked credentials, and compromised digital accounts to make rapid unauthorized purchases. If these fraudulent activities are not detected immediately, innocent consumers suffer financial disruption, merchants lose merchandise, and financial institutions incur severe chargeback expenses.

Traditional fraud detection systems typically rely on static rules or overnight batch processing. Batch analysis examines transaction logs hours or even days after financial settlement has already taken place. By the time an alert is generated, the stolen funds have often been withdrawn and cannot be easily recovered. Meanwhile, simple static rules fail to adapt to individual consumer habits, often causing false alarms that decline valid transactions from honest customers.

Sentinel AI solves these shortcomings through an adaptive, low-latency monitoring pipeline. When a new transaction occurs, the system analyzes the payment within milliseconds. Instead of relying on a single technique, Sentinel AI evaluates the transaction using three complementary perspectives: a trained machine learning model, an individual cardholder spending baseline, and seven deterministic business fraud rules.

Importantly, Sentinel AI operates as an intelligent decision-support assistant rather than an automated bank payment switch. It does not freeze actual bank accounts or halt live banking settlement networks. Instead, it calculates an explainable composite risk score, generates a human-readable explanation of why the charge looks suspicious, and records the event in an investigation portal. This approach ensures that human investigators retain full visibility and final authority over all fraud decisions.

OBJECTIVES

The primary objective of the Sentinel AI project is to design, implement, and validate an end-to-end real-time financial fraud intelligence system. The specific technical and functional goals achieved by the implementation are outlined below:

- **Low-Latency Event Streaming:** Ingest continuous financial transaction streams using Apache Kafka to ensure immediate event availability with zero processing delays.
- **Unified Canonical Feature Pipeline:** Extract standardized transaction features—including amount, time of day, merchant category codes, and geographic locations—using identical logic for offline model training and real-time inference.
- **Personalized Behavioral Profiling:** Maintain sliding causal memory windows (one minute, ten minutes, and twenty-four hours) per cardholder to capture transaction frequency and detect spending anomalies without future data leakage.
- **Supervised Machine Learning Inference:** Benchmark and deploy a trained Random Forest classification model to recognize complex multidimensional fraud patterns with high precision.
- **Unsupervised Anomaly Exploration:** Benchmark Isolation Forest models on transaction distributions to evaluate unsupervised anomaly detection on novel attack vectors.
- **Deterministic Business Logic Enforcement:** Implement and enforce seven explicit fraud rules to capture well-known fraud behaviors, including rapid burst spending, impossible geographic travel, and unfamiliar merchant charges.
- **Composite Risk Scoring & Explainability:** Synthesize machine learning predictions (50%), behavioral baselines (30%), and business rules (20%) into a weighted fraud-risk score accompanied by plain-English audit reasons.
- **Decoupled Data Persistence:** Store raw immutable transactions, prediction outputs, investigation records, and audit logs across decoupled relational tables in PostgreSQL.
- **Investigator Case Management Portal:** Provide a secure web dashboard where fraud analysts can review flagged transactions, examine score breakdowns, and update investigation decisions.
- **Cardholder Self-Service Security:** Provide a customer-facing interface where users can view their personal transaction history and submit dispute inquiries for unrecognized charges.

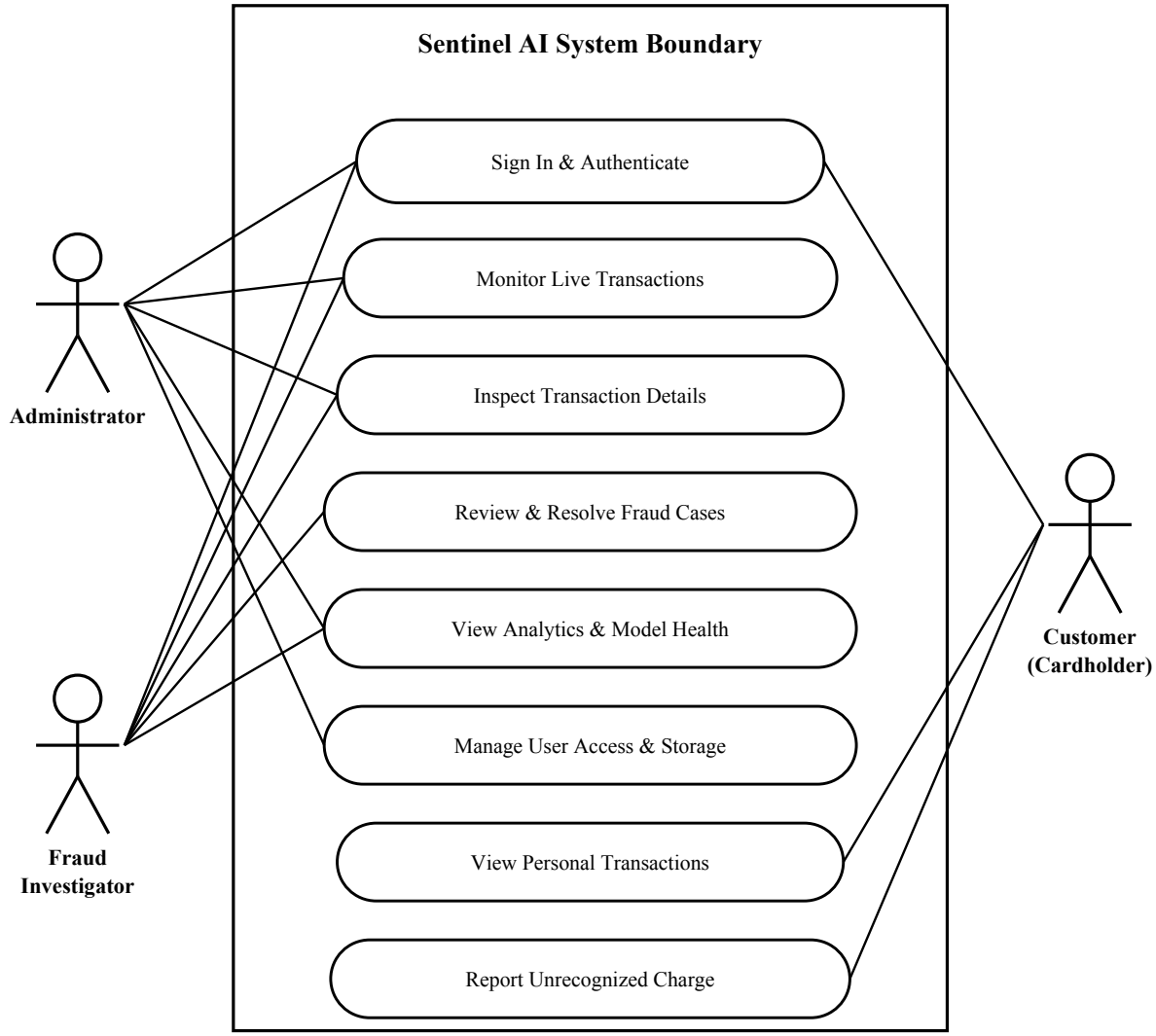
PROBLEM STATEMENT

In modern financial systems, electronic payment cards and digital wallets process hundreds of millions of transactions daily. Safeguarding this massive transaction volume against organized fraud presents several fundamental challenges that existing commercial and academic solutions fail to resolve adequately:

- **Impossibility of Manual Review:** The overwhelming volume of transactions makes manual inspection by human analysts impossible. Without automated detection, unauthorized transactions easily remain undetected until an affected cardholder discovers unexpected charges days or weeks later.
- **Rigidity of Global Static Rules:** Many legacy fraud detection systems rely entirely on simple threshold rules (such as flagging any single purchase above five hundred dollars). These static rules ignore individual context: a five-hundred-dollar purchase may be routine for a frequent business traveler, but completely abnormal for a college student. This rigidity generates high rates of false positives, inconveniencing legitimate cardholders.
- **Opacity of Black-Box Machine Learning:** Advanced neural networks and tree-based machine learning classifiers can calculate statistical fraud probabilities, but they rarely explain why a particular transaction was flagged. When a fraud analyst must resolve a disputed payment or contact a merchant, an unexplained risk score creates operational bottlenecks and regulatory compliance risks.
- **Latency of Batch Processing:** Many existing financial architectures still rely on batch processing jobs that execute overnight or every few hours. In fast-moving fraud scenarios—such as automated card-testing scripts—criminals execute dozens of fraudulent charges within seconds. Discovering these charges hours later prevents timely mitigation and leads to significant financial loss.

How Sentinel AI Addresses These Problems: Sentinel AI resolves these limitations through an adaptive hybrid architecture. By buffering transactions in Apache Kafka and processing micro-batches with Apache Spark, the system evaluates each charge within milliseconds. By weighting a supervised machine learning model (50%), a personalized behavioral baseline (30%), and transparent business rules (20%), Sentinel AI achieves both high detection accuracy and low customer friction. Furthermore, the system outputs clear, plain-language audit explanations alongside every score, empowering investigators to make informed decisions without delay.

USE CASE DIAGRAM



The diagram above depicts the formal UML use case architecture implemented in Sentinel AI. The system defines three primary human actors with strictly enforced role-based permissions: **Administrator**, **Fraud Investigator**, and **Customer**. The Administrator oversees system operations, manages user accounts, and inspects audit logs. The Fraud Investigator monitors the live transaction feed, examines multidimensional risk factors, and updates investigation case outcomes. The Customer is restricted by database Row-Level Security to viewing only their own card transactions and submitting dispute tickets for unrecognized charges. The system acts as a secure analytical platform and does not perform direct banking payment holds.

SYSTEM REQUIREMENTS

The requirements listed below reflect the actual software components, frameworks, and hardware configurations verified in the Sentinel AI project codebase and container deployment configuration:

Component / Layer	Verified Technology & Specifications	Category
Host Operating System	macOS (Darwin ARM64 / Apple Silicon or Intel) or Linux (Ubuntu 20.04+ LTS)	Mandatory
Container Platform	Docker Desktop (v4+) and Docker Compose (v2+) for service containerization	Mandatory
Core Backend Language	Python 3.10+ (specifically validated on Python 3.13 virtual environment)	Mandatory
Streaming Message Broker	Apache Kafka 3.7.0 in KRaft mode (Topic: transactions-canonical, 3 partitions)	Mandatory
Stream Processing Engine	Apache Spark / PySpark (v4.2 / 3.5) with Java OpenJDK 17+ runtime	Core Streaming
Relational Database	PostgreSQL 15 (Docker container sentinel_postgres) or Supabase Cloud with RLS	Mandatory
Web Application API	FastAPI with Uvicorn, Pydantic data schemas, and JWT authentication	Mandatory
Primary Web Frontend	Next.js 15 (React 19, TypeScript, Tailwind CSS, Lucide icons, port 3000)	Mandatory
Analytics Dashboard	Streamlit interactive KPI dashboard (dashboard/app.py)	Optional
Machine Learning Stack	Scikit-learn (Random Forest, Logistic Regression, Isolation Forest), NumPy, Pandas	Mandatory

Hardware Specifications (Established from Deployment Configuration):

• **Processor (CPU):** Modern 64-bit multi-core processor (minimum 2 physical cores, 4 recommended) to execute Kafka, PostgreSQL, FastAPI, and Spark containers concurrently.

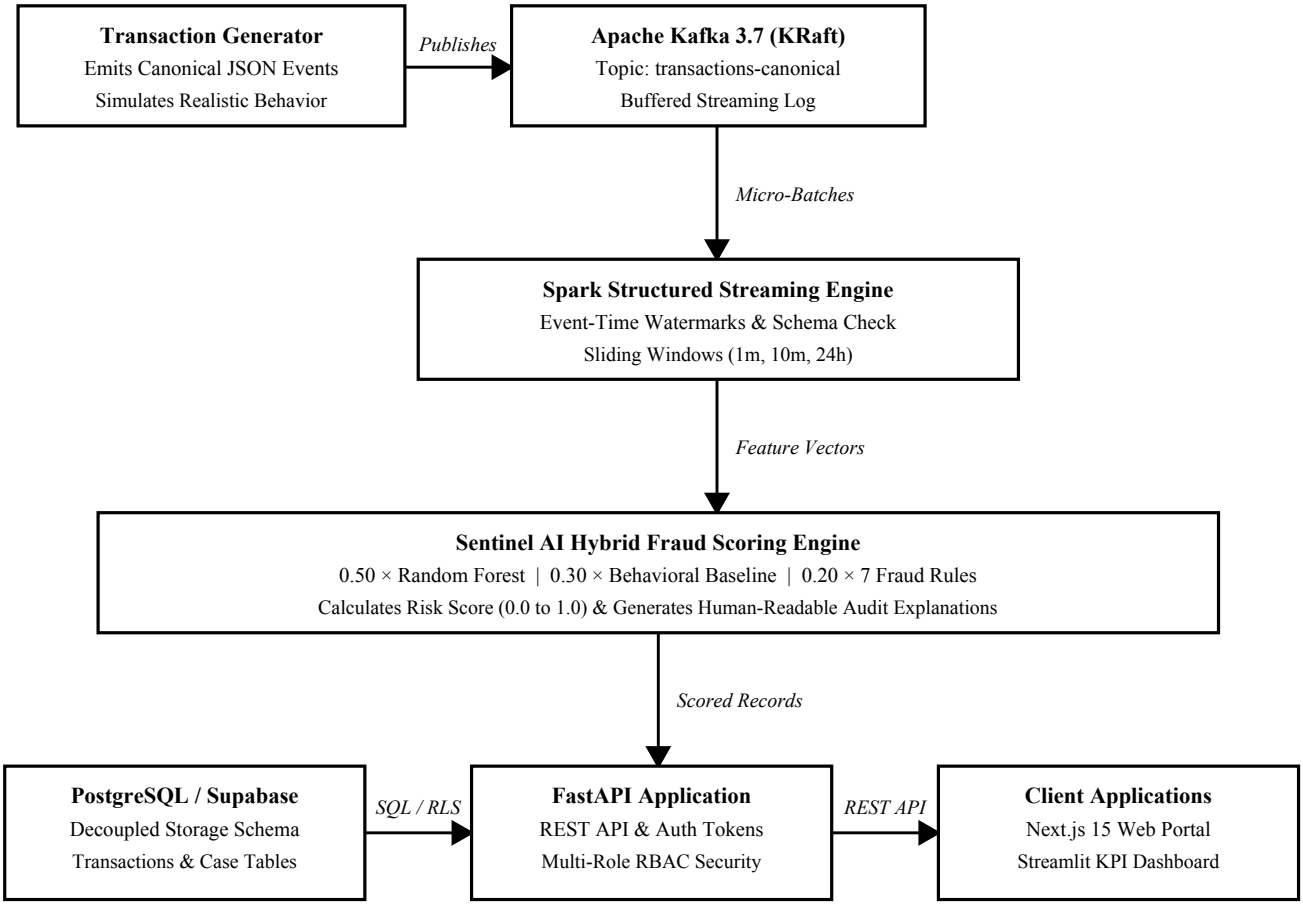
• **System Memory (RAM):** Minimum 4 GB RAM (8 GB recommended). The Spark streaming worker is explicitly allocated 2 GB of driver memory (SPARK_DRIVER_MEMORY: 2g), with remaining RAM hosting Docker containers and application services.

• **Storage Capacity:** Minimum 5 GB of free disk space for Docker image layers, Parquet training datasets, model pickle binaries, and PostgreSQL data volumes.

• **Networking:** Standard local loopback network interface (127.0.0.1) supporting internal inter-container communication across designated ports (8000 for FastAPI, 3000 for Next.js, 9092 for Kafka, and 5433 for PostgreSQL).

ARCHITECTURE DESIGN

Sentinel AI is architected as an event-driven, decoupled system that isolates ingestion, stream processing, machine learning inference, database persistence, and user interfaces into modular components:



Component Interaction and Data Flow: The transaction generator emits canonical JSON events to Apache Kafka, buffering high-volume traffic. Apache Spark Structured Streaming consumes events in micro-batches and derives temporal velocity features across 1-minute, 10-minute, and 24-hour windows. The hybrid scoring engine combines the supervised Random Forest model (50% weight), the personalized cardholder baseline (30% weight), and seven deterministic business rules (20% weight) to calculate a composite risk score (0.0 to 1.0). Scored records and opened cases are saved to decoupled tables in PostgreSQL or Supabase Cloud. The FastAPI backend enforces role-based access control and serves secured REST endpoints to the Next.js web portal and Streamlit monitoring dashboard.

METHODOLOGY

Sentinel AI processes each transaction through an eight-stage pipeline that guarantees immediate evaluation, consistency between training and serving, and human-interpretable reasoning:

- **Step 1: Event Ingestion & Kafka Buffering** — When a transaction occurs, the transaction payload is validated against a canonical event schema and published to the Apache Kafka topic (transactions-canonical). Kafka partitions buffer high-throughput traffic, preventing downstream processing bottlenecks.
- **Step 2: Canonical Feature Extraction** — The canonical feature extractor computes standardized attributes, including purchase amount, log-transformed amount, cyclic time representations (hour sine and cosine), day of week, merchant category risk weight (MCC), and transaction channel risk weight (e.g., in-person chip vs. online ecommerce).
- **Step 3: Temporal Windowing & Velocity Calculation** — The streaming engine checks the cardholder's recent activity across sliding causal windows: a 1-minute window to identify rapid burst transactions, a 10-minute window for short-term spending velocity, and a 24-hour window for daily volume deviation, strictly avoiding future data leakage.
- **Step 4: Supervised Machine Learning Inference** — The pre-trained Random Forest classifier processes the normalized feature vector. It evaluates multidimensional nonlinear correlations and outputs an objective fraud probability score between 0.0 and 1.0.
- **Step 5: Behavioral Profiling & Rule Evaluation** — The transaction is compared against the cardholder's historical baseline to calculate the amount z-score and identify unfamiliar merchants. Concurrently, the deterministic rule engine checks seven explicit business rules, including impossible geographic travel, burst transactions, and unusual operating hours.
- **Step 6: Hybrid Risk Score Calculation** — The composite fraud score is computed using the fixed weighted formula: Final Score = (0.50 × ML Score) + (0.30 × Behavior Score) + (0.20 × Rule Score). The score maps to three decision categories: Legitimate (below 0.40), Suspicious (0.40 to 0.69), or Fraud (0.70 and above).
- **Step 7: Decoupled Database Persistence** — Scored records are committed to PostgreSQL or Supabase Cloud across decoupled tables (transactions, fraud_predictions, and fraud_investigations). If a transaction is classified as Suspicious or Fraud, an investigation case is automatically opened.
- **Step 8: Alert Presentation & Case Management** — Flagged transactions appear on the investigator dashboard with transparent, plain-English audit reasons (such as 'Amount is 12.7x customer average' or 'Impossible travel detected'). Analysts can review the evidence, inspect telemetry, and record final fraud determinations.

FUTURE SCOPE

While the current Sentinel AI prototype demonstrates an end-to-end working system for real-time transaction monitoring, several important enhancements could be pursued in future engineering phases. The planned future extensions described below are clearly distinguished from the current software implementation:

- **Evaluation on Real-World Commercial Banking Data:** The current machine learning models were benchmarked on the public ULB credit card fraud dataset and synthetic streaming scenarios. Future research should evaluate the system on real-world, non-anonymized transaction streams from commercial banking partners to validate performance against emerging attack techniques.
- **Automated Investigator Feedback & Continuous Retraining:** While the current system captures investigator review decisions (confirmed fraud vs. false positive) in PostgreSQL, model retraining is performed offline. A valuable future extension is an active learning pipeline that schedules automatic micro-batch model retraining using confirmed investigator labels.
- **Direct Banking Gateway Integration:** Sentinel AI currently operates as an analytical decision-support tool. In future production banking deployments, the scoring engine could interface directly with core banking authorization switches (such as Visa, Mastercard, or SWIFT) to place pre-authorization holds on suspicious transactions before funds are settled.
- **Automated Multi-Channel Customer Alerts:** While cardholders can currently inspect charges and submit disputes via their web portal, integrating external notification services (such as automated SMS verification, email alerts, or mobile push notifications) would enable cardholders to confirm or dispute suspicious charges immediately on their smartphones.
- **Graph-Based Fraud Syndicate Detection:** Adding a graph database engine (such as Neo4j) would allow the system to detect organized fraud rings where multiple fraudulent cards share the same device fingerprint, shipping address, or foreign IP address across disparate accounts.
- **Distributed Cloud Cluster Deployment:** The current architecture is containerized for single-host Docker Compose deployment. Transitioning to a managed Kubernetes cluster with auto-scaling Spark worker pods and multi-node Kafka clusters would provide enterprise-grade fault tolerance and high-volume redundancy.

CONCLUSION

Sentinel AI successfully demonstrates a comprehensive, real-time financial fraud intelligence system developed as an academic B.Tech Computer Science project. By synthesizing distributed message streaming, big data micro-batch processing, supervised machine learning, individualized behavioral profiling, and deterministic business rules, the project provides a scalable and transparent solution to the critical challenge of financial transaction monitoring.

The actual implementation proves that high-throughput financial transactions can be ingested and evaluated within milliseconds. By combining a supervised Random Forest classifier (50%), a personalized cardholder baseline (30%), and seven explicit business fraud rules (20%), Sentinel AI achieves strong fraud detection capabilities while significantly reducing false alarms for legitimate customers. Rather than producing opaque, unexplained risk numbers, the system generates human-readable audit explanations for every alert, providing immediate clarity during investigator reviews.

As an academic software engineering effort, Sentinel AI bridges the gap between big data infrastructure (Apache Kafka, Apache Spark, PostgreSQL/Supabase) and modern web applications (FastAPI, Next.js). The platform enforces robust security principles, including role-based access control, PostgreSQL Row-Level Security, and strict separation between customer and investigator portals.

It is important to state the honest limitations of the project: Sentinel AI is an analytical decision-support and investigative monitoring platform, not an operational banking payment switch that directly freezes customer cards or settles live bank transfers. Its evaluation relies on public benchmark datasets and simulated streaming transactions rather than proprietary commercial banking feeds.

In conclusion, Sentinel AI demonstrates that transparent, hybrid artificial intelligence combined with distributed big data streaming provides an effective, ethical, and explainable foundation for modern financial transaction intelligence.